

Adaptive Retrieval-Augmented Generation via Group Relative Policy Optimization

Optimizing Models for Dynamic Retrieval

Wenlong Zheng¹ Axel Tang¹ Zihan Gong¹ Tanish Upreti¹

¹The Edward S. Rogers Sr. Department of Electrical and Computer Engineering

Applied Deep Learning – Fall 2025

Introduction: The Need for Adaptive RAG

Problem: Static RAG Inefficiency

Traditional RAG retrieves documents once regardless of query complexity. This leads to either **insufficient context** for hard queries or **redundant retrieval** for simple ones.

- **Objective:** Train a small, efficient router model (Qwen2.5-0.5B) to dynamically decide *when* to retrieve, *what* to ask, and *when* to stop.
- **Task:** Given a natural language question, the router must iteratively:
 - 1 propose sub-questions for retrieval,
 - 2 decide whether more retrieval is needed,
 - 3 and finally output a `<terminate>` signal.
- **Methodology:** Group Relative Policy Optimization (GRPO).
- **Why GRPO?**
 - Eliminates the need for a Critic Network.
 - Uses group-based relative reward optimization without human preference labeling.



Methodology: Adaptive RAG Architecture Setup

We employ a **Reinforced Retrieval-Generation Pipeline** optimized for the 4090 GPU environment.

Models

- **Router (Actor):** Qwen2.5-0.5B-Instruct (trained).
- **Reference Policy:** Frozen Qwen2.5-0.5B for KL penalty.
- **Answer Generator:** Qwen2.5-7B generating final answers used for reward.

Retrieval Setup

- **Corpus:** English Wikipedia corpus (June 2024 dump - 42M documents).
- **Retriever:** BGE-M3 dense retriever.
- **Index:** FAISS IVF65536 + PQ128 for memory efficiency.

Implementation Details: Exploration & Optimization

- **Step-wise termination exploration:** Increase the probability of forcing a `<terminate>` token from 2% to 50% across steps to encourage early stopping when confident.
- **Multi-step trajectory stitching:** Construct single-sequence inputs from multi-turn history (User → Assistant → Retrieval) for log-probability calculation.

Custom Reward Design

- **Termination & Outcome (~60%):** Dominant incentive for "Perfect Stops" to prioritize efficiency.
- **Format Compliance (~20%):** Dense rewards for strict XML syntax validation (e.g., `<subtopic>`) to stabilize instruction-following capabilities.
- **Sub-question Relevance (~20%):** Integrates a **Reward Model** to score query quality, preventing the agent from "hacking" format rewards with meaningless text.

Preliminary Results & Future Work

Current Achievements:

- Complete Adaptive RAG pipeline implemented.
- Successful convergence of the 0.5B model using GRPO.
- Model learned to generate valid search queries and stop appropriately.

Performance Metric

The 0.5B model achieves comparable routing performance to larger 7B model baselines but with **1/14th the parameter count**, significantly reducing inference costs.

Conclusion: Adaptive RAG via GRPO proves that small language models can effectively master tool-use and dynamic retrieval through rigorous Reinforcement Learning, provided that exploration strategies (noise) and reward functions are carefully engineered.